

BÁO CÁO KẾT QUẢ HUẤN LUYỆN MỚI: Kích thước cửa sổ (v32), Knowledge Distillation và v30_optimize

Mục tiêu: Tổng hợp và phân tích các thí nghiệm huấn luyện mới nhất trên bộ dữ liệu SisFall cho bài toán nhận diện hành vi và phát hiện té ngã chạy on-device trên ESP32-S3, gồm ba hướng: (i) khảo sát ảnh hưởng của kích thước cửa sổ (v32 — w128/w256); (ii) thử nghiệm Chung cất tri thức (Knowledge Distillation) từ TCN sang CNN; và (iii) cải tiến kiến trúc head giữ trực thời gian, cho ra mô hình triển khai đề xuất v30_optimize.

Quy ước số liệu: “offline FLOAT” = đánh giá model Keras float trên tập test desktop (subject-independent); “firmware INT8” = chạy model INT8 thật trên ESP32-S3 (240 MHz) qua bộ kiểm thử riêng 1000 mẫu (200 mẫu/lớp). Hai bộ test khác nhau nên chỉ so sánh firmware-vs-firmware hoặc offline-vs-offline.

1. Bối cảnh và Pipeline chung

- **Phân loại 5 nhãn:** Walk, Run, Idle, Trans, Fall (Fall = index 4). Chỉ số ưu tiên số 1 là **Fall recall**, luôn báo kèm **Trans F1**; không kết luận bằng accuracy tổng (lớp đa số Walk/Run/Idle chiếm ~78% mẫu).
- **Tiền xử lý:** downsample 200 Hz → 100 Hz; chuẩn hóa accel clip($\pm 8g$)/8, gyro clip($\pm 500dps$)/500 (khớp firmware); cắt cửa sổ theo sự kiện; augment lớp Trans bằng nhân biên độ $\times 0.9/\times 1.1$; class-weight Fall $\times 3$, Trans $\times 0.55$; ngưỡng đánh giá fall_threshold = 0.25.
- **Ràng buộc phần cứng (ESP-NN):** chỉ các toán tử được tăng tốc mới dùng cho nhánh tốc độ (Conv 1 $\times$ 1, depthwise, MaxPool, FullyConnected, mean/GAP, relu6). **Không** tăng tốc: LSTM/GRU, dilated conv (dilation > 1), sigmoid/tanh.

2. Thí nghiệm Kích thước Cửa sổ (v32: w128 và w256)

Mục tiêu của nhóm thí nghiệm v32 là kiểm chứng giả thuyết: liệu chọn kích thước cửa sổ là lũy thừa của 2 (128, 256) có giúp tăng tốc suy luận trên ESP32-S3 hay không. Cả hai phiên bản dùng chung kiến trúc CNN thuần 6 trục như v30, chỉ thay đổi độ dài cửa sổ đầu vào.

2.1. Kết quả offline FLOAT

Lưu ý: số mẫu tập test thay đổi theo độ dài cửa sổ (cửa sổ ngắn → nhiều cửa sổ hơn) nên cột “Số mẫu” khác nhau giữa các phiên bản.

Phiên bản	Cửa sổ	Số mẫu test	Accuracy	F1-Fall	F1-Trans	Macro-F1	Fall Recall
v30 (mốc tham chiếu)	200	6.106	0.9343	0.9859	0.7993	0.9259	97.60%
v32_w128	128	9.233	0.9272	0.9764	0.7240	0.9069	96.53%
v32_w256	256	4.780	0.9354	0.9919	0.8359	0.9302	98.53%

2.2. Kết quả firmware INT8 (ESP32-S3, 1000 mẫu)

Phiên bản	Cửa sổ	Latency	Tensor Arena	Kích thước INT8	Accuracy	F1-Trans	Fall Recall
v32_w128	128	13.05 ms	12.484 B	~24.5 KB	0.9340	0.8811	97.00%
v30	200	19.78 ms	16.508 B	~25.0 KB	0.9290	0.8703	98.00%
(200)							
v32_w256	256	25.07 ms	19.652 B	~24.5 KB	0.9270	0.8617	98.00%

2.3. Nhận xét

- **Độ trễ tỷ lệ tuyến tính với độ dài cửa sổ:** $13,05/19,78 \approx 0,66$ và $25,07/19,78 \approx 1,27$, khớp gần đúng với tỷ lệ độ dài $128/200 = 0,64$ và $256/200 = 1,28$.
- **Kết luận then chốt:** việc chọn cửa sổ là lũy thừa của 2 **không** mang lại lợi thế tăng tốc nào. Hiệu ứng cân chỉnh bộ nhớ theo lũy thừa của 2 chỉ phát huy ở chiều **số kênh** (channel — chiều trong cùng được SIMD đóng gói), không áp dụng cho chiều thời gian (vốn chỉ là số vòng lặp tích chập).
- Cửa sổ 256 mẫu cho độ chính xác offline nhỉnh hơn đôi chút nhưng chậm hơn ~27% trên firmware; cửa sổ 128 nhanh nhất nhưng F1-Trans offline sụt mạnh (0.7240) do thiếu ngữ cảnh thời gian. Kích thước 200 mẫu vẫn là điểm cân bằng hợp lý.

3. Hướng Knowledge Distillation (KD) — Negative Result

3.1. Ý tưởng và cách làm

Quan sát thấy CNN thuần nhanh nhưng yếu ở Idle/Trans, còn TCN giỏi Idle/Trans nhưng quá chậm (2058 ms, không deploy được), một hướng tự nhiên là **Chưng cất tri thức**: giữ student = CNN v30 nguyên bản (nhanh), cho học nhãn mềm (soft-label) từ teacher TCN (giỏi Trans). Hàm mất mát có dạng $\alpha \cdot \text{CE}_{\text{class-weight}} + (1 - \alpha) \cdot T^2 \cdot \text{KD}$.

Đã thử **4 công thức**: KD cross-entropy (T=4); KD KL-divergence (T=2); và selective per-class trust (tin teacher ít ở Fall, nhiều ở Trans).

3.2. Kết quả (offline FLOAT)

Cấu hình	Idle F1	Trans F1	Fall Recall	Accuracy	Kết luận
Baseline CNN v30 (không KD)	~0.910	0.799	97.60%	0.9343	Mốc so sánh
KD-KL (T=2)	~0.909	0.818	97.47%	0.9329	Không thắng baseline

Cấu hình	Idle F1	Trans F1	Fall Recall	Accuracy	Kết luận
KD-CE / selective (3 bản còn lại)	~0.909	~0.80–0.82	~97.x%	< 0.9343	Idle F1 đồng bằng

Kết quả chung: Idle F1 đồng bằng ở mức ~0.909 và Trans precision kẹt ~0.75 qua **mọi** công thức. Không bản KD nào vượt được baseline.

3.3. Chẩn đoán nguyên nhân

1. **Teacher TCN yếu hơn baseline ở nhãn Fall** ($97.07\% < 97.60\%$) → chung cất toàn cục kéo Fall của student đi xuống.
2. **Head GAP+GMP của CNN xóa trực thời gian** → student “mù” với *vị trí/hình dạng* của transition trong cửa sổ. Đây là **trần kiến trúc**, không phải trần công thức huấn luyện.

⇒ **Bài học:** KD không thể vá một giới hạn kiến trúc. Phải sửa kiến trúc của student.

4. Cải tiến Kiến trúc (KD2 → v30_optimize)

4.1. Mạch tư duy thiết kế: chất lọc nguyên lý từ TCN

Cải tiến kiến trúc không phải “thử ngẫu nhiên” mà rút ra từ việc *vì sao TCN mạnh ở Idle/Trans rồi mang nguyên lý đó sang CNN bằng cơ chế rẻ hơn*. Chuỗi suy luận gồm bốn bước:

1. **Quan sát:** TCN đạt Trans 0.927 / Idle 0.950 (cao nhất) trong khi CNN baseline chỉ 0.799 / 0.910. Điều TCN làm được mà CNN baseline không: **mô hình hóa trình tự/hình dạng theo thời gian** của tín hiệu (mỗi nơ-ron “thấy” cả pattern *yên* → *bùng phát* → *yên*).
2. **Rào cản:** chính dilated conv đó **không được ESP-NN tăng tốc** (2058 ms). Tức *cơ chế* của TCN (dilation) bị cấm trên phần cứng, **nhưng nguyên lý (giữ cấu trúc thời gian) thì không**.
3. **Bài học:** phân biệt Idle/Trans bắt buộc phải **giữ thông tin thời gian** — head GAP+GMP gộp phẳng trực thời gian nên đánh mất đúng thứ này.
4. **Chuyển nguyên lý sang CNN bằng cơ chế ESP-NN-friendly:** (a) mở receptive field bằng strided/pooled downsampling (thay dilation); (b) head Flatten giữ layout thời gian ở map thô 12 bước (thay vì gộp GAP/GMP).

Đối chiếu cơ chế — cùng mục tiêu, khác phương tiện:

Tiêu chí	TCN (gốc)	CNN cải tiến (v30_optimize)
Mục tiêu	Giữ & mô hình hóa trình tự thời gian	(giống)
Mở receptive field	Dilated conv (d=1,2,4,8)	Strided conv + MaxPool (200 → 12)
Giữ layout thời gian ở classifier	Conv giữ chiều thời gian đến cuối	Flatten map thô 12×96
Tương thích ESP-NN	Không — dilation chạy reference → 2058 ms	Có — toàn op tăng tốc → 56.7 ms

Tiêu chí	TCN (gốc)	CNN cải tiến (v30_optimize)
Trans / Idle F1	0.927 / 0.950	0.907 / 0.942 ($\approx$ TCN, nhanh $\sim 36\times$)

4.2. Ba thay đổi cụ thể so với CNN v30 baseline

1. **Head mới (quyết định):** GAP + GMP + Flatten (giữ bản đồ đặc trưng thô 12×96) thay cho GAP+GMP. Nhánh Flatten giữ thông tin *chuyển động xảy ra ở đoạn thời gian nào* — chính cái baseline bị mù (GAP/GMP vẫn giữ để robust với Fall).
2. **Sâu hơn:** 2 lớp SeparableConv ($k=3$) mỗi tầng (thay vì 1) ở các tầng đầu.
3. **Rộng hơn:** số kênh $24/32/48/64 \rightarrow 32/48/64/96$.

	CNN v30 baseline	v30_optimize
SeparableConv mỗi tầng	1	2 (ở 2 tầng đầu)
Số kênh	24/32/48/64	32/48/64/96
Head	GAP+GMP (128-d)	GAP+GMP+ Flatten (1.344-d)
Số tham số	$\sim 7.4k$	$\sim 26.6k$

4.3. Kiến trúc layer-by-layer của v30_optimize

Khối	Thành phần	Output shape
Đầu vào	—	(200, 6)
Stem	Conv1D(32, $k=3$, $s=2$) + BN + ReLU6	(100, 32)
Block 1	SepConv1D(48, $k=3$) $\times 2$ + ReLU6 + MaxPool(2)	(50, 48)
Block 2	SepConv1D(64, $k=3$) $\times 2$ + ReLU6 + MaxPool(2)	(25, 64)
Block 3	SepConv1D(96, $k=3$) + ReLU6 + MaxPool(2)	(12, 96)
Head	GAP GMP Flatten $\rightarrow$ Concat (1.344) $\rightarrow$ Dropout $\rightarrow$ Dense(5)	(5,)

4.4. Ablation: KD có thực sự cần không? (KHÔNG)

Huấn luyện kiến trúc mới với nhiều mức α ($\alpha = 1.0$ = tắt KD hoàn toàn). Offline FLOAT:

Biến thể (kiến trúc mới)	Idle F1	Trans F1	Fall F1	Fall Recall	Accuracy
KD $\alpha=0.5$ (single-TCN)	0.942	0.906	0.986	97.20%	0.9532

Biến thể (kiến trúc mới)	Idle F1	Trans F1	Fall F1	Fall Recall	Accuracy
KD $\alpha=0.3$ (teacher-heavy)	0.942	0.917	0.986	97.20%	0.9551
$\alpha=1.0$ (TẮT KD)	0.943	0.914	0.985	97.47%	0.9563
Ensemble teacher (TCN+ResNet+CNN)	0.943	0.912	0.987	97.33%	0.9571
fall_recall_boost (Fall w $\uparrow$)	0.941	0.907	0.989	97.73%	0.9558

Kết luận: càng ít KD càng tốt; bản $\alpha=1.0$ (không thầy) ngang bằng hoặc hơn mọi bản có KD. **Toàn bộ thắng lợi đến từ KIẾN TRÚC (head giữ-trục-thời-gian), không phải distillation.** → bỏ teacher, đóng gói thành v30_optimize huấn luyện supervised thuần. (Phù hợp lý thuyết: teacher không đủ vượt trội + capacity gap khiến soft-label thậm chí gây nhiễu nhẹ.)

4.5. Tinh chỉnh cuối: monitor val_accuracy thay val_loss

Với model có class-weight, val_loss (không trọng số) thiên về lớp đa số → chọn epoch kém ở lớp thiểu số. Đổi sang monitor='val_accuracy':

v30_optimize	Idle F1	Trans F1	Fall Recall	Accuracy
monitor = val_loss	0.9379	0.8944	97.47%	0.9530
monitor = val_accuracy	0.9420	0.9071	97.73%	0.9533

⇒ Accuracy tổng gần như đứng yên (đa số che lấp) nhưng **Trans +0.013** và **Fall recall +0.26** (vượt baseline 97.60%). Bản val_accuracy là bản deploy cuối.

5. Kết quả Mô hình Triển khai Đề xuất: v30_optimize

5.1. Chi tiết offline FLOAT (tập test 6.106 mẫu, threshold 0.25)

Nhãn	Precision	Recall	F1-Score
Walk	0.9712	0.9365	0.9535
Run	0.9814	0.9724	0.9769
Idle	0.9273	0.9572	0.9420
Trans	0.8857	0.9295	0.9071

Nhãn	Precision	Recall	F1-Score
Fall	0.9879	0.9773	0.9826
Tổng	—	—	Acc 0.9533 / Macro-F1 0.9524

Fall Recall = 97.73% (733/750 ca ngã được phát hiện đúng).

5.2. So sánh trên firmware INT8 (ESP32-S3)

Mô hình	Accuracy	Idle F1	Trans F1	Fall F1	Fall Recall	Latency	Arena	tfllite
CNN v30 base-line	0.920	0.859	0.853	0.980	96.00%	20 ms	16.5 KB	25 KB
TCN (teacher)	0.952	0.922	0.936	0.980	97.00%	2058 ms	50 KB	105 KB
v30_optimize	0.953	0.926	0.945	0.987	97.50%	56.7 ms	28.3 KB	55.8 KB

Nhận xét:

- Nén INT8 gần như **không suy hao**: offline float acc 0.9532 → firmware INT8 0.9530.
- v30_optimize trên chip **bằng/vượt teacher TCN** (acc 0.953 vs 0.952; Trans 0.945 vs 0.936) nhưng **nhANH hơn ~36 lần** (56.7 ms vs 2058 ms).
- So baseline CNN v30 trên chip: **Trans +9.2, Idle +6.7, Fall recall +1.5**.
- Latency 56.7 ms hoàn toàn phù hợp real-time: suy luận theo cửa sổ (~chu kỳ 500 ms) → hệ số chiếm dụng CPU chỉ ~11%.

6. Kết luận và Đề xuất

- v30_optimize là mô hình triển khai đề xuất**: đạt độ chính xác ngang TCN (acc 95.3%, Trans F1 0.945 trên firmware) mà vẫn thuần toán tử ESP-NN, độ trễ 56.7 ms — khả thi cho thiết bị đeo chạy pin. Đóng góp cốt lõi là **head GAP+GMP+Flatten** chắt lọc nguyên lý mô hình hóa thời gian của TCN vào một CNN nhẹ.
- Knowledge Distillation là một negative result có giá trị**: teacher TCN không đủ vượt trội + capacity gap; chuyển *nguyên lý kiến trúc* hiệu quả hơn nhiều so với chuyển *tri thức qua soft-label*.
- Window power-of-2 không giúp tăng tốc**: độ trễ tỷ lệ tuyến tính với độ dài cửa sổ; căn chỉnh lũy thừa của 2 chỉ có lợi ở chiều số kênh.
- Kỷ luật đánh giá**: mọi kết luận dựa trên Fall recall + Trans F1, không dùng accuracy tổng (bị lớp đa số che lấp); và đổi monitor huấn luyện sang val_accuracy để chọn đúng epoch tốt cho lớp thiểu số.

7. Hạn chế và việc còn lại

- **Cần k-fold subject-independent để xếp hạng tin cậy:** chênh lệch giữa các biến thể ($\sim 0.4\%$ acc, $\text{Trans} \pm 0.02$) nằm trong khoảng nhiễu của một lần chia (single-split), do tập test nhân Trans chỉ có 567 mẫu. Khung k-fold (5 kịch bản S1_Elderly / S2_Young / S3-S4 cross / S5_Both) đã có sẵn để chạy thẳng cho v30_optimize (không cần teacher vì KD đã chứng minh không giúp).
 - **Nhất quán pipeline khi k-fold:** bản k-fold hiện dùng `decimate(q=2)` khác với `iloc[:,2]` của bản deploy — cần thống nhất trước khi đổi chiều số liệu.
 - **Ứng viên tối đa hóa Fall recall:** bản `fall_recall_boost` đạt Fall recall offline 97.73% nhưng **chưa được kiểm chứng trên firmware** — là lựa chọn dự phòng nếu ưu tiên tuyệt đối việc không bỏ sót té ngã.
-

8. Tham chiếu file (repo SisFall-PreProcessing/)

- Thí nghiệm cửa sổ: `train_v32_w128/`, `train_v32_w256/` (kèm `report_*_sram_firmware.txt`).
- KD thất bại (trên CNN cũ): `train_v30_kd/`.
- Kiến trúc mới + ablation các α : `train_v30_kd2/{kd_alpha_0_5, kd_alpha_1_0_ablation, kd_alpha_0_3_teacher_heavy, fall_recall_boost, ensemble_teacher_alpha_0_5}/`.
- Mô hình deploy cuối: `train_v30_optimize/val_accuracy/`.
- Tài liệu chi tiết hướng v30_optimize: `docs/BAO CAO TRIEN KHAI_v30_optimize.md`.